Universidad Mariano Gálvez De Guatemala, Sede Boca Del Monte

Facultad Ingeniería en Sistemas

Docente: Henrry Sontay

Guatemala Noviembre 2026

Proyecto Final

Alumno: Edwin Geovanni García Ortega

7691-23-14677

1. Introducción

En este proyecto proponemos desarrollar Greninja Adventure, un videojuego bidimensional de plataformas inspirado en los juegos clásicos. Buscamos aplicar los conocimientos de Arquitectura II programando directamente el microcontrolador Arduino Due mediante lenguaje ensamblador ARM. El jugador podrá desplazarse, saltar, evitar enemigos y acumular puntos. Este documento describe el diseño previsto; no representa una validación física ya realizada.

2. Objetivos

Objetivo general: desarrollar un videojuego 2D mediante ensamblador ARM y acceso directo a registros del Arduino Due.

Objetivos específicos: configurar GPIO, ADC y SPI; leer el joystick y los botones; implementar gravedad, salto y colisiones; mostrar gráficos, puntuación y vidas; y utilizar un temporizador con interrupciones NVIC.

3. Hardware propuesto

Componente	Función
Arduino Due SAM3X8E	Procesamiento ARM Cortex-M3 a 84 MHz
TFT ILI9341 SPI 240 × 320	Presentación de gráficos a color
Joystick KY-023	Movimiento horizontal y lectura de ejes
Botón Z integrado	Iniciar y reiniciar
Pulsador externo	Saltar
Protoboard, cables y USB	Montaje, alimentación y programación

La pantalla TFT fue autorizada como alternativa a OLED/VGA. Se debe verificar el controlador, la interfaz SPI y las especificaciones eléctricas del módulo comprado.

4. Mapa de memoria

Región	Dirección base	Tamaño / uso
Flash interna	0x00080000	512 KB: instrucciones y datos constantes
SRAM0	0x20000000	64 KB: variables, pila y buffers
SRAM1	0x20080000	32 KB: almacenamiento adicional
Periféricos	0x40000000 (región)	Registros mapeados en memoria

Las dos regiones SRAM son separadas; el linker script debe definirlas expresamente. La ubicación final de cada símbolo se verificará con el archivo .map generado durante la compilación.

4.1 Secciones de memoria

Sección	Contenido
.isr_vector	Vectores de interrupción
.text	Código ensamblador
.rodata	Sprites y fuentes constantes
.data	Variables inicializadas
.bss	Variables sin inicializar
Stack	Pila de funciones e interrupciones

4.2 Variables del juego

Variable	Función
estado_juego	Menú, jugando o Game Over
greninja_x / greninja_y	Posición del personaje
velocidad_y	Velocidad vertical y gravedad
puntos / vidas	Marcador y supervivencia
enemigo_x	Posición de enemigo
frame_tick	Marca de actualización periódica

5. Máquina de estados

MENÚ: muestra el título y espera el botón Z. JUGANDO: lee entradas, aplica físicas, detecta colisiones y actualiza el marcador. GAME OVER: suspende la actualización de físicas, muestra el fin de partida y espera el botón Z para reiniciar.

5.1 Diagrama de flujo



6. Conexiones electrónicas

6.1 Pantalla TFT ILI9341 SPI

Pantalla	Arduino Due
VCC	3.3 V solo si el módulo admite esa alimentación
GND	GND común
SCK	SCK del conector SPI central de 6 pines
MOSI / SDI	MOSI del conector SPI central
CS	D10
DC / RS	D9
RST	D8
MISO / SDO	MISO del conector SPI, opcional
LED / BL	Según ficha técnica del módulo

6.2 Joystick y pulsador

Señal	Arduino Due
Joystick VCC / GND	3.3 V / GND
Joystick VRx / VRy	A0 / A1
Joystick SW	D2, con pull-up
Pulsador externo	D3 a GND, con pull-up

El Due utiliza lógica de 3.3 V: no aplicar 5 V a sus entradas. Todas las tierras deben estar conectadas. Antes de encender, confirmar el pinout real de la TFT y su retroiluminación.

7. Temporización por hardware

Se propone actualizar las físicas a 60 Hz mediante un temporizador de hardware y una interrupción atendida por NVIC. El período es T = 1 / 60 = 16.67 ms. Si el reloj efectivo del temporizador es 42 MHz, el valor teórico de comparación sería RC = 42,000,000 / 60 = 700,000. Debe confirmarse la fuente de reloj, el modo de contador y los registros del SAM3X8E antes de utilizar este valor.

8. Cálculo de refresco de pantalla

Resolución: 240 × 320 = 76,800 píxeles. En RGB565 se transmiten 2 bytes por píxel: 153,600 bytes por cuadro completo. A 30 FPS: 153,600 × 30 = 4,608,000 bytes/s = 36.864 Mbit/s, sin sobrecarga.

Con SPI ideal a 42 Mbit/s, un cuadro completo requiere 153,600 × 8 / 42,000,000 ≈ 29.26 ms. Como 30 FPS equivalen a 33.33 ms por cuadro, queda poco margen para comandos, lógica y dibujo. Se propone redibujar regiones modificadas y medir el rendimiento real en la placa. No se garantiza todavía una tasa de 30 FPS.

9. Organización modular del código

Archivo	Responsabilidad
startup.s	Arranque y vectores
main.s	Máquina de estados
gpio.s / adc.s / buttons.s	Pines, joystick y botones
spi.s / tft.s	Transferencia SPI y pantalla
timer.s	Temporizador e interrupciones
sprites.s / physics.s	Gráficos, gravedad y salto
collision.s / hud.s	Colisiones, puntos y vidas
memory.ld	Regiones de memoria y enlace

Los módulos funcionales se implementarán en archivos .s sin bibliotecas gráficas ni API Arduino.

10. Plan de pruebas

Prueba	Resultado esperado
Encendido	Aparece el menú
Botón Z	Inicia o reinicia
Joystick	Mueve al personaje
Pulsador D3	Activa salto
Gravedad	Regresa a la plataforma
Enemigos y monedas	Colisiones y puntuación correctas
Game Over	Detiene físicas y permite reinicio
Timer y TFT	Cadencia y refresco medidos

Estado de pruebas: pendientes. No marcar como aprobadas hasta realizarlas con el hardware y documentar los resultados.